

Teola: 迈向 LLM 应用的端到端优化

Xin Tan¹, Yimin Jiang², Yitao Yang¹, Hong Xu¹

¹The Chinese University of Hong Kong, ²Unaffiliated

摘要

基于大语言模型 (LLM) 的应用由 LLM 与非 LLM 组件共同组成, 各组件都影响端到端时延。尽管已有大量工作优化 LLM 推理, 端到端工作流优化仍受到忽视。现有框架以任务模块开展粗粒度编排, 将优化限制在各模块内部, 导致次优调度决策。

我们提出细粒度端到端编排, 以任务原语为基本单位, 将每个查询的工作流表示为原语级数据流图。它显式暴露更大的设计空间, 允许不同模块的原语进行并行化与流水线优化, 并改进调度, 提升应用级性能。我们构建 Teola, 一个实现此方案的新型 LLM 应用编排框架。广泛实验表明, 在多类流行应用上, Teola 相对于现有系统最高获得 2.09 倍加速。代码地址为 <https://github.com/NetX-Lab/Ayo>。

1 引言

大语言模型 (LLM) 及其多模态变体改变了用户查询理解和内容生成, 推动传统与新兴应用发展。例如, 一些搜索引擎将 LLM 集成到查询处理管线中, 改善用户体验 [4, 18]。此外, 人工智能智能体作为新的人机交互范式, 催生了情感陪伴 [5] 和个性化助手 [21] 等应用。

虽然 LLM 是应用中最智能的组件, 仅靠它通常无法满足多样而复杂的需求。例如, 受模型设计限制, 知识时效性和长上下文理解表现仍有不足; 处理不当时, 容易出现幻觉 [40]。为缓解这些问题, 已有检索增强生成 (RAG) [49, 57]、外部函数调用 [16, 41, 47] 及多模型交互等技术。LangChain [12]、LlamaIndex [1] 等框架支持整合不同模块, 构建上述端到端管线。

虽然已有工作从多个方面优化 LLM 推理 [25, 30, 48, 76, 80], 由多种模块组成的应用, 其端到端性能仍少受关注。图 1 展示使用 LlamaIndex [1] 的流行应用执行时间构成, 非 LLM 模块占显著比例, 某些 RAG 文档问答场景甚至超过 50%。然而, 在现有框架 [1, 10, 12, 17] 中优化整体性能, 比预想更困难。它们将工作流组织为简单的模块链 (图 3a), 各模块独立、顺序执行高层任务, 使用各自引擎, 如 LLM 推理使用 vLLM [48]。这种粗粒度链式方式虽然易用, 却把模块当作黑盒, 限制跨模块联合优化 (§2.2)。前端编排与后端执行解耦, 也使调度只能优化单请求, 无法优化应用整体, 甚至降低总效率 (§2.3)。

本文主张用更细粒度方式暴露并编排 LLM 应用, 为端到端优化奠定基础。我们以原语级数据流图替代模块链, 使用任务原语作为基本单位。每个原语是执行特定操作的符号节点, 并携带存储关键属性的元数据 (§2.2)。图中显式的属性与交互, 使系统可优化整个工作流, 寻找端到端时延更低的执行计划 (§2.2)。数据流图还捕获不同原语请求的关联、依赖和拓扑深度, 支持面向应用的调度与批处理 (§2.3)。

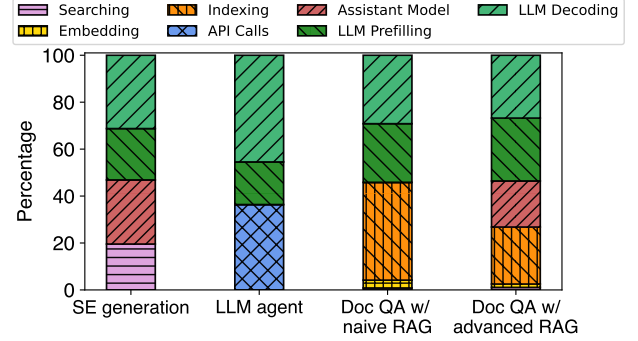


图 1. 图 2 各应用使用 LlamaIndex [1] 时的模块时延构成; LLM 答案合成分为预填充与解码。Percentage 为占比, Searching / Embedding / Indexing 为搜索 / 嵌入 / 索引, API Calls 为 API 调用, Assistant Model 为辅助模型, LLM Prefilling / Decoding 为 LLM 预填充 / 解码。横轴为搜索辅助生成、LLM 智能体、朴素 RAG 文档问答与高级 RAG 文档问答。

据此, 我们构建基于原语的 Teola 编排框架, 包含两个主要组件: (1) 图优化器, 结合查询输入与配置, 以及开发者预定义的粗粒度工作流, 为每个用户查询构建原语级数据流图, 再通过针对性的优化步骤产生高效执行图; (2) 运行时调度器, 采用两层机制, 上层调度各查询执行图, 下层由各引擎调度器管理, 将请求同一引擎的原语合批处理, 利用这些请求的关系实现应用感知调度。

我们主要使用 Ray [61] 实现分布式调度与执行, 并借助多种库实现执行引擎。在多个数据集及应用上评估 Teola, 包括搜索辅助生成、朴素与高级 RAG 文档问答, 以及 Anthropic 新发布的生产工作流 [6]。平台实验表明, 相对于现有方案, 端到端时延最高获得 2.09 倍加速。比较方案包括我们用 Ray 构建的 LlamaIndex [1] 分布式实现、进一步加入模块并行与 LLM 执行优化的增强版, 以及智能体框架 AutoGen [72]。

本文贡献如下:

- 我们识别现有编排框架的局限: 粗粒度模块编排限制优化潜力, 请求级调度目标与应用端到端性能不匹配。
- 我们提出以原语级数据流图表示查询工作流的细粒度编排, 扩大端到端优化空间, 支持并行化、流水线等图优化及应用感知调度。
- 我们设计并实现 Teola, 验证该方法的可行性与收益。流行 LLM 应用实验显示, 其性能优于当前比较系统。

2 背景与动机

2.1 基于 LLM 的应用

LLM 基础。当前 LLM 建立在 Transformer 上, 依赖注意力机制捕获自然语言中的长上下文 [70]。本文关注的推

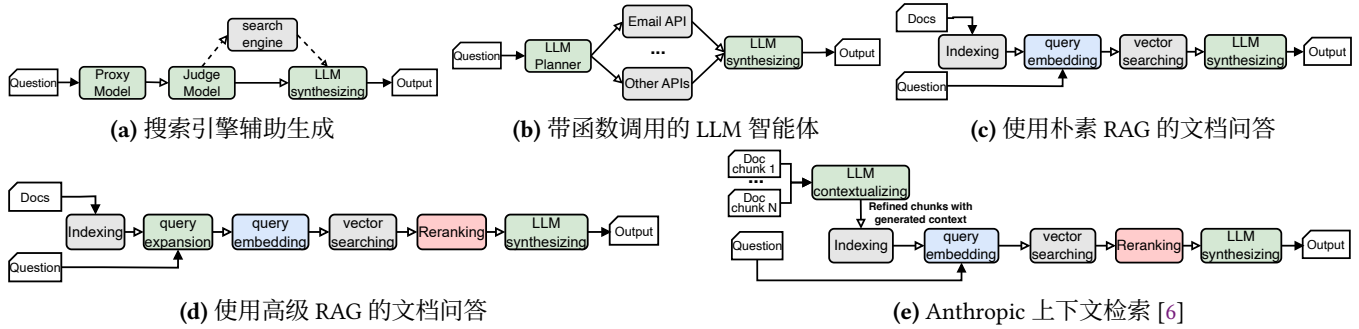


图 2. 从编排框架 [1, 12, 17] 和 Anthropic [6] 获得的常见实际应用 workflow。Question / Docs / Output 为问题 / 文档 / 输出，Proxy / Judge Model 为代理 / 判定模型，search engine 为搜索引擎，LLM Planner 为模型规划器，Email / Other APIs 为邮件 / 其他 API，LLM synthesizing 为模型答案合成；Indexing 为索引，query expansion / embedding 为查询扩展 / 嵌入，vector searching 为向量搜索，Reranking 为重排。Doc chunk 为文档片段，LLM contextualizing 为模型上下文化，Refined chunks with generated context 为附加生成上下文后的片段。

理是自回归的：每次前向计算产生一个新词元，它作为语言建模基本单位，进入上下文并用于后续迭代输入。为避免重复计算先前词元的注意力相关数据，推理使用键值（KV）缓存，它也成为重要内存压力来源 [48, 76]。

推理分为预填充与解码。预填充处理指令、上下文等全部输入词元，生成首个输出词元，通常受计算能力限制。随后解码利用 KV 缓存逐步生成剩余输出，每轮只处理上一轮新生成的词元，通常受内存带宽限制。

LLM 应用还需要其他组件。 尽管生成能力强，LLMs 并非万能。训练数据不可避免地缺少最新知识，可能出现知识缺口与幻觉 [10, 49]；它也不能直接与环境交互，例如能起草邮件却不能直接发送 [38, 64, 72]。实际应用因而需要整合额外工具。

图 2 展示五种 workflow，其中四种广泛用于企业生产和开源项目，一种是 Anthropic 近期发布的生产 workflow。图 2a 的搜索辅助生成使用搜索引擎，回答超出模型知识范围的问题 [4, 44, 66]，由代理模型与判定模型决定是否调用搜索。图 2b 的通用智能体通过不同工具 API 执行计划，例如利用用户账号权限起草并发送邮件 [38, 72]。图 2c 与图 2d 分别为朴素与高级 RAG 文档问答。RAG 可能是最流行的 LLM 增强技术，已有大量生产应用 [10, 17]：将用户上传文档分片，经过嵌入模型处理后，写入向量数据库作为领域知识库 [10, 17, 49]，这一过程称为索引；再检索相关片段供 LLM 生成回答。高级版本（图 2d）用 LLM 扩展和细化查询 [32, 43]，提高搜索准确性，重排各扩展查询检索出的片段，合成精准答案。最后，图 2e 为 Anthropic 上下文检索 [6]：LLM 利用文档其他内容为每片生成上下文摘要，并将摘要加到片段前，使其单独处理时仍与原文保持一致。

我们还进行了小规模案例研究，在 GitHub 使用“LLM applications”检索，选取最匹配且超过 1000 个 star 的 30 个项目，包括 Haystack [11]、AutoGen [72] 和 LangChain [12]。我们手工检查是否包含四类 workflow 或相似模式；一个项目常包含多类。表 1 显示这些模式很普遍，大多数出现在超过一半样本中，RAG 问答最常见。

workflow	数量	占比
搜索辅助生成	16	53.33%
LLM 智能体	13	43.33%
朴素 RAG 文档问答	26	86.67%
高级 RAG 文档问答	23	76.67%

表 1. 在 30 个超过 1000 个 star 的 GitHub 样本项目中，四类 workflow（图 2(a)-(d)）的出现数量和比例。各类别并不互斥，同一项目可包含多个 workflow。

随着 workflow 广泛使用，如 §1 所述，复杂应用中 LLM 已不一定是唯一瓶颈。因此，精心编排各组件非常重要。

2.2 LLM 应用的细粒度编排

LlamaIndex [1]、LangChain [12]、PAI-RAG [17] 和 Azure-RAG [10] 等框架与企业方案支持构建应用，通常使用模块级编排，把应用定义为简单模块链，如图 3a 所示；各模块在后端独立执行。这虽然易实现和使用，却限制复杂 workflow 优化，忽视联合分析模块内部操作依赖这一更大设计空间。一些智能体框架 [3, 72] 更紧凑、粗粒度地连接承担不同子任务的角色，以预定义 workflow 完成复杂任务，同样未充分探索跨智能体或子任务的端到端优化。

本文核心主张是更细粒度地暴露与编排 LLM 应用，改善整体性能。图 3a 的同一 workflow 可表示为图 3b：将各模块拆为作为图节点的原语。例如，索引拆为嵌入生成与数据写入，查询扩展和答案合成拆为预填充与解码。

数据流图显式捕获原语依赖，使跨原语、跨模块的复杂联合优化成为可能。例如，图 3b 中嵌入生成与写入可和查询扩展的预填充、解码并行，因为输入相互独立。各原语的输入输出关系与其他关键信息，例如 §2.3 的示例，编码为节点属性。

随后，我们优化原语图，寻找整个 workflow 端到端时延更低的计划。图 3c 为图 3b 对应的优化执行图。查询扩展生成多个新查询，因此可把解码拆为多个部分解码原语，逐个输出查询，立即送入下游嵌入原语，无需等待

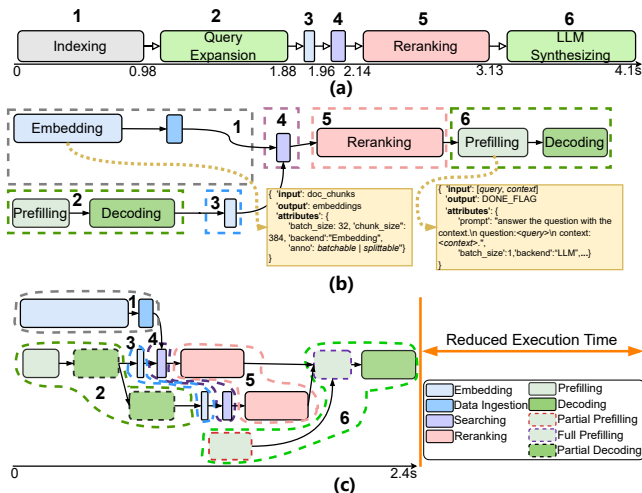


图 3. 现有方案与 Teola 的工作流表示及执行比较。(a) 现有模块级工作流，箭头仅表示执行顺序，不表示数据依赖；(b) Teola 的原语数据流图，因篇幅仅显示部分元数据，箭头表示原语间依赖；(c) 优化后的执行图。Indexing 为索引，Query Expansion 为查询扩展，Embedding 为嵌入，Data Ingestion 为数据写入，Searching 为搜索，Reranking 为重排，LLM Synthesizing 为答案合成；Prefilling / Decoding 为预填充 / 解码，Partial Prefilling / Full Prefilling 为部分 / 剩余完整预填充，Partial Decoding 为部分解码。Reduced Execution Time 表示节省的执行时间，从 4.1 秒缩短至 2.4 秒。元数据 input / output 为输入 / 输出，attributes 为属性，prompt 为提示词，batch_size / chunk_size 为批量 / 片段大小，backend 为执行后端，anno 为注解；例如嵌入批量为 32、片段大小为 384，LLM 批量为 1，提示词意为“依据上下文回答问题；问题：query；上下文：context”。DONE_FLAG 为完成标记，batchable / splittable 为可合批 / 可拆分。

全部查询生成。同理，答案合成的预填充可先处理系统指令和用户问题，在搜索与重排前，与索引并行运行。

原语级数据流图因此暴露了现有模块编排无法看到的并行与流水线机会。收益显著：示例图 3 的执行时间由 4.1 秒缩短至 2.4 秒。

2.3 应用感知的调度与执行

现有编排的另一局限是后端引擎优化请求级性能，与用户感知的应用级性能不匹配。假设使用常见的 Triton [22] 服务索引模块的嵌入模型，如图 4a，它把请求均匀处理，固定批量为 4；没有应用信息，只能在合理但次优 GPU 利用率下优化单请求时延。如果知道请求来自同一模块，便应优化全部请求完成时间，使用例如 16 的更大批次充分利用 GPU。共 48 个片段对应 48 次请求时，虽然每批稍慢，总时间却从 1.8 秒降至 1.35 秒，获得约 1.3 倍加速（图 4a）。

这一示例利用单个原语内部请求的关联。另一类信息是跨原语依赖。图 4b 的树式答案合成涉及连续多次 LLM 调用；传统请求级调度以批量 2 优化各请求时延。若知道它们构成深度为 2 的依赖树，就可按相同深度合

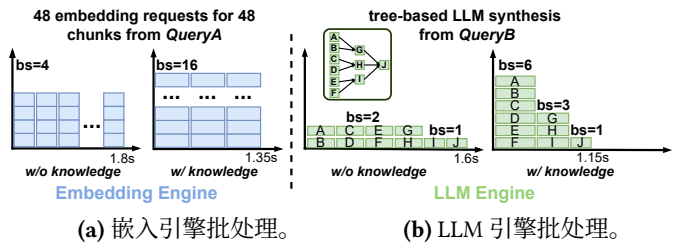


图 4. 请求级与应用级调度执行比较。左图为 QueryA 的 48 个片段的 48 次嵌入请求，bs 为批量大小，Embedding Engine 为嵌入引擎；w/o knowledge 为没有关联信息，w/ knowledge 为有信息，批量从 4 调到 16，完成时间从 1.8 秒降至 1.35 秒。右图为 QueryB 的树式 LLM 答案合成，LLM Engine 为模型引擎；按树深度改变批量大小，总完成时间从 1.6 秒降至 1.15 秒，字母表示请求节点。

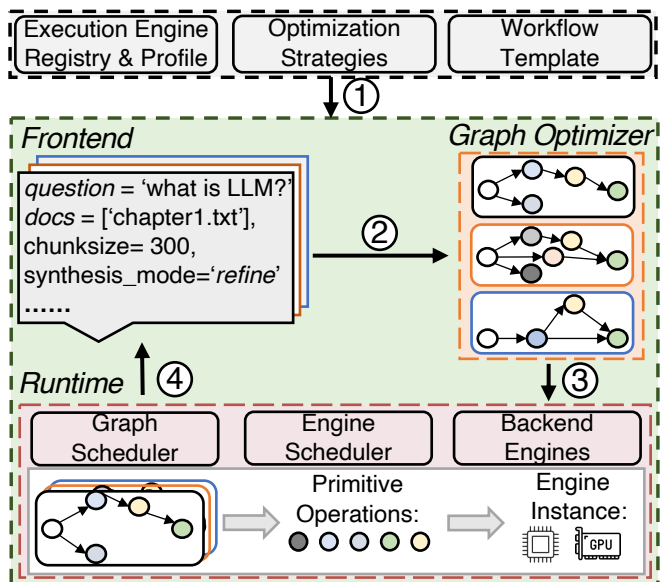


图 5. Teola 系统概览。Execution Engine Registry & Profile 为执行引擎注册与性能档案，Optimization Strategies 为优化策略，Workflow Template 为工作流模板，Frontend 为前端，Graph Optimizer 为图优化器，Runtime 为运行时，Graph / Engine Scheduler 为图 / 引擎调度器，Backend Engines 为后端引擎，Primitive Operations 为原语操作，Engine Instance 为引擎实例。前端示例查询为“什么是 LLM？”，文档为 chapter1.txt，片段大小 300，答案合成模式为 refine（逐步改进）。① 配置，② 构图优化，③ 提交执行，④ 返回结果。

批、使用不同批量大小，即使每批时延稍大，仍获得约 1.4 倍整体加速。

因此，细粒度编排利用原语图节点属性中的请求关联和依赖，弥合引擎请求级优化与应用性能之间的差距，进一步优化端到端执行。

```

1 from Teola.app import APP, Node
2 from Teola.executor import Engine
3 from Teola.graph import OPT_Pass, Graph
4 # 注册执行引擎, 例如 LLM。
5 LLM=Engine("LLM", executable=llm_exec, config=config_llm,
6           resource={"GPU":2}, instances=2)
7 # 注册优化步骤。
8 OPT_Pass.register("pipelining", pipeline_pass)
9 app=APP.init() # 初始化应用。
10 # 注册带配置说明的模板组件。
11 query_expand=Node("LLM", in_kwargs, out_kwargs,
12                  anno="splittable", config=config_expand)
13 embedding=Node("Embedding", in_kwargs, out_kwargs,
14               anno="batchable", config=config_embed)
15 # 省略其他组件。
16 generation=Node("LLM", in_kwargs, out_kwargs,
17                 anno=None, config=config_gen)
18 # 声明依赖关系。
19 query_expand>>embedding>>...>>generation
20 # 更新应用工作流模板。
21 app.update_template([query_expand, ...])
22 # 依据查询和配置构建并优化图。
23 e_graph=Graph.optimize(app, query, config, OPT_Pass)
24 # 提交运行时并调度。
25 e_graph.schedule()

```

代码 1. Teola 的简化使用示例。

3 设计概览

3.1 架构

Teola 是以原语操作为基本单位、优化 LLM 应用执行的新型编排框架。

图 5 展示架构。在离线阶段 ①，开发者注册应用使用的嵌入模型、LLM 和数据库操作等引擎，同时提供不同输入大小下的时延档案，例如批量大小与序列长度。他们还给出包含组件及执行顺序的模板，如查询扩展和 LLM 生成，类似现有框架的任务模块 [1, 12]；也可为某些原语指定优化策略。完成配置和部署后，系统即可在线服务。

在线阶段收到带输入数据和配置的查询后，Teola 构建原语数据流图 (p -graph) ②，应用相关优化得到执行图 (e -graph)，提交运行时 ③。运行时准确跟踪执行状态，把原语高效调度到合适后端，完成后向前端返回结果 ④。¹

3.2 API

代码 1 为简化使用示例，主要组件如下。

执行引擎。引擎处理 workflow 组件的模型或操作请求，见代码第 5 行，可分无模型与基于模型两类。数据库等无模型引擎主要使用 CPU，不涉及 DNN；模型引擎可部署 BERT 系列 [31] 等嵌入模型或生成用 LLM。一个引擎可服务用途不同的多个组件，例如图 2d 的查询扩展与答案合成共用同一 LLM。

工作流定义。Teola 隐藏图优化和调度复杂性，但开发者仍需为每个应用先定义带运行时占位符的高层模板，与现有框架 [1, 2, 12, 72] 类似。利用 API 的第 8–20 行，指定组件的引擎、职责及输入输出配置。可标注 batchable，表示输入独立、适于合批；或 splittable，表示输出可

类型	说明
重排 (Reranking)	计算查询与上下文对的相关分数并排序。
写入 (Ingestion)	将嵌入向量写入向量数据库
搜索 (Searching)	在数据库中执行向量搜索
嵌入 (Embedding)	为文档或问题生成嵌入向量
预填充 (Prefilling)	LLM 推理的预填充部分
解码 (Decoding)	LLM 推理的解码部分
部分预填充	计算提示词的部分前缀，例如指令、上下文或问题。
剩余完整预填充	部分预填充之后，处理提示词剩余部分
部分解码	对完整输出的一部分执行解码
条件 (Condition)	决定条件分支
聚合 (Aggregate)	汇总多个原语的结果

表 2. 图 2d 中的原语示例。白底为常规操作，蓝底为拆分操作，灰底为控制流操作。

拆分。>> 运算符指定执行顺序，确保数据流正确。模板为详细执行图的构建与优化提供基础，使其适应查询配置。²

图优化。每个查询结合具体数据、配置与模板构建更细的原语节点 p -graph，见第 22 行。Teola 用内置优化步骤匹配原语操作与模式，识别高效计划，生成 e -graph (§4)。开发者还可通过接口注册自定义优化，见第 7 行。

声明式查询。用户通过声明式接口向已部署应用提交查询。除问题与上下文外，还可定制工作流 (图 5)，例如调整索引片段大小、LLM 提示词模板与答案合成模式，满足性能需求。

4 图优化器

图优化器结合查询信息与工作流模板，为每个查询生成细粒度表示，即 p -graph。图由符号原语节点构成，再通过优化策略产生高效的执行图 e -graph。

4.1 原语图 p -graph

原语。如 §2.2 所述，只依赖高层组件，会过度简化操作之间的复杂关系，暴露的信息和灵活性有限。我们提出任务原语，简称原语，类似 TensorFlow [23] 的操作节点。工作流级符号原语提高表示粒度，并在执行前提供优化所需信息。

如表 2 所示，原语可以对应注册引擎中的标准操作，例如嵌入生成、上下文重排，也可以对应拆分后的细粒度操作。LLM 推理分为 Prefilling 与 Decoding；Partial Prefilling 和 Full Prefilling 共同构成预填充，Partial Decoding 处理完整解码的不同部分。还可有 Aggregate 和 Condition 等聚合与条件分支控制原语。各原语具有输入、输出、父子节点等元数据，构成图的基础，同时携带 DNN 批量大小、LLM 提示词和目标执行引擎等关键属性。

构建 p -graph。优化器将原模板 $\mathcal{T} = (\mathcal{T}_N, \mathcal{T}_E)$ ，结合查询配置 $C = (\mathcal{T}_N, C_N)$ ，转换为更细的 $\mathcal{G} = (\mathcal{V}_N, \mathcal{V}_E)$ ，见算法 1。 $\mathcal{T}_N$ 为组件， $\mathcal{T}_E$ 为依赖， C_N 为用户配置。系统依据配置，把各模板组件拆成明确的符号原语，形成依赖清晰的子图。例如，具有三个上下文片段、采用 refine 模式的答案合成，会转换为三对串联的 Prefilling 与

¹译注：p-graph 中的 p 是 primitive (原语)，并不表示 probabilistic (概率)；本文没有在此定义概率 DAG 模型。

²译注：代码示例写作 splittable，正文使用 splittable；pipelining 的函数标识符写为 pipeline_pass。这些代码拼写按原文保留。

算法 1 图转换与优化

```

1: function GRAPHTRANSFORM( $\mathcal{T}, C$ )
2:    $\mathcal{V}_N \leftarrow \{\}; \mathcal{V}_E \leftarrow \{\}$  ▷ 原语集合及其数据依赖
   # 依据配置将各模板组件拆成原语子图,
   # 并维护子图依赖。
3:   对 每个  $t \in \mathcal{T}_N$  执行
4:      $Prims, Edges \leftarrow \text{DecomposeComponent}(t, C)$ 
5:      $Prims \leftarrow \text{Configure}(Prims, C)$ 
6:      $\mathcal{V}_N.\text{extend}(Prims); \mathcal{V}_E.\text{extend}(Edges)$ 
   # 维护模板原有的组件依赖。
7:   对 每个  $(t_i, t_j) \in \mathcal{T}_E$  执行
8:      $tailp \leftarrow \text{GetTailPrim}(t_i); headp \leftarrow \text{GetHeadPrim}(t_j)$ 
9:      $\mathcal{V}_E.\text{append}((tailp, headp))$ 
10:  返回  $\mathcal{G}_p = (\mathcal{V}_N, \mathcal{V}_E)$  ▷ 返回原语图  $p\text{-graph}$ 
11: function GRAPHOPT( $\mathcal{G}_p, \mathcal{P}$ )
   #  $\mathcal{G}_p$  为原语图,  $\mathcal{P}$  为执行引擎性能档案。
12:   $\mathcal{G}_e \leftarrow \text{PrunDependency}(\mathcal{G}_p)$  ▷ 优化步骤 1
13:   $\mathcal{G}_e \leftarrow \text{StageDecompose}(\mathcal{G}_e, \mathcal{P})$  ▷ 优化步骤 2
14:   $\mathcal{G}_e \leftarrow \text{PrefillingSplit}(\mathcal{G}_e)$  ▷ 优化步骤 3
15:   $\mathcal{G}_e \leftarrow \text{DecodingPipelining}(\mathcal{G}_e)$  ▷ 优化步骤 4
16:  返回  $\mathcal{G}_e$  ▷ 返回优化执行图  $e\text{-graph}$ 

```

Decoding 原语, 配置各自元数据。最终图保留原工作流程依赖, 同时更详细地揭示内部操作。

4.2 优化

如 §2.2 所述, Teola 聚焦扩大分布式执行并行度; 单项操作加速与此相互独立, 详见 §9。优化器识别原语间并行及流水线并行机会, 采用一组静态、基于规则的优化。

可利用的机会。原模板继承的依赖只表示高层执行顺序, 在细粒度图中可能有冗余。分析并剪除不必要依赖, 可释放独立原语, 形成并行数据流分支, 扩大并行度 (**步骤 1**)。计算密集原语也可在可行时拆成多个流水阶段, 与下游同时执行 (**步骤 2**)。

LLM 还具有两项特殊性质: (1) 因果预填充允许将预填充拆成相互依赖的部分, 让部分预填充与上游其他原语并行 (**步骤 3**); (2) 流式解码输出使某些自回归解码的部分结果可提前作为下游输入, 产生新的流水机会 (**步骤 4**)。

优化步骤。基于上述分析, 我们在原语图中整合如下步骤, 优化端到端执行:

- ▷ **步骤 1: 依赖剪枝。**检查各原语输入与当前上游原语的关系, 删除不必要依赖, 识别可并发的数据流分支。剩余边只表示真实数据依赖, 某些原语因此脱离原模板的依赖结构。例如, 图 3c 中的查询扩展与嵌入原语拆开, 形成新分支。
- ▷ **步骤 2: 阶段拆分。**对 batchable 原语, 若数据超过引擎最大高效批量大小, 即继续增加批量已不提升吞吐量, 则拆成多个阶段, 各处理一个子微批次, 与下游可合批原语形成流水线。更细划分可能提高流水并行度, 但寻找最优粒度耗时, 相邻可合批原语还会造成指数级搜索空间, 不适合时延敏感场景。为平衡资源利用与执行效率, 我们只在输入大小达到最大高效批量时显式分段; 必要时, 流水线末尾添加 Aggregate, 同步并汇总结果。

- ▷ **步骤 3: 拆分 LLM 预填充。**完整提示词包含系统/用户指令、问题和上下文等部分。某些部分提前可用, 如用户指令和问题; 另一些还未就绪, 如 RAG 数据库检索上下文。无需等全部就绪, 在遵守注意力因果性的前提下, 可提前处理已就绪的部分, 使部分预填充与其他操作并行。³

- ▷ **步骤 4: LLM 解码流水线。**词元逐步生成, 一旦形成语义完整的部分, 例如查询扩展中的一句改写查询, 就立即送入下游可合批原语, 不必等待全部解码。该调用需标注 splittable, 表示输出可按语义拆分; 相应解析器监控逐渐生成的 JSON 等结构化输出, 提取完整部分, 随即传给后继。

优化过程。优化器迭代遍历原语图, 将节点与各优化模式匹配; 匹配成功便应用步骤, 修改相关原语, 见算法 1, 持续到没有进一步可用优化。可缓存优化后的子图以减少开销。⁴

示例。图 6 展示图 2d 应用优化后的执行图。查询扩展生成三个新查询, 提高检索效果; 前三个相关片段进入答案合成。合成采用 refine 模式: 先用排名第一片段和问答提示词生成候选答案, 再用其余两片段及改进提示词修订两次。各优化步骤已在图中标出。

5 运行时调度

Teola 使用两层运行时调度。上层图调度器派发每个查询优化执行图的原语节点; 下层引擎调度器管理引擎实例, 并融合不同查询的原语请求以提高执行效率。图调度与操作执行分离, 提升可扩展性与可扩充性。

5.1 图调度器

图调度器持续跟踪各执行图状态, 依赖满足后派发原语。它检查节点入度, 入度降为零时发给对应引擎调度器。派发的是原语节点本身, 而非单独请求, 从而使下层知道哪些请求源于同一原语, 与现有框架独立处理请求不同 (§2.3)。原语完成后, 通过 RPC 通知调度线程并传输输出; 线程减少下游节点入度, 使其准备执行。

每个查询还具有专属对象存储来管理中间结果, 既为等待原语保存输入, 也提供一定的容错能力, 减少操作失败造成的影响。

5.2 引擎调度器

专属引擎调度器管理各实例, 使映射到不同引擎类型的原语独立执行。主要挑战是高效融合请求同一引擎的原语。优化后的查询可能同时派发多个节点, 也可能在引擎队列中积累多个等待节点, 尤其是多个组件共用引擎

³译注: 因果约束要求提前计算的是有效提示词前缀。不能跳过尚未就绪的前缀, 任意预填充其后的片段, 也不能在不考虑提示词次序的情况下直接拼接 KV 缓存。

⁴译注: 代码标识符保留原文。GraphTransform / GraphOpt 为图转换 / 优化, DecomposeComponent 为组件拆分, Configure 为配置元数据, GetTailPrim / GetHeadPrim 为取得尾 / 首原语, extend / append 为扩展 / 追加集合; PrunDependency、StageDecompose、PrefillingSplit、DecodingPipelining 分别实现依赖剪枝、阶段拆分、预填充分拆和解码流水线。本文采用规则与启发式, 不提供全局最优执行计划的证明。

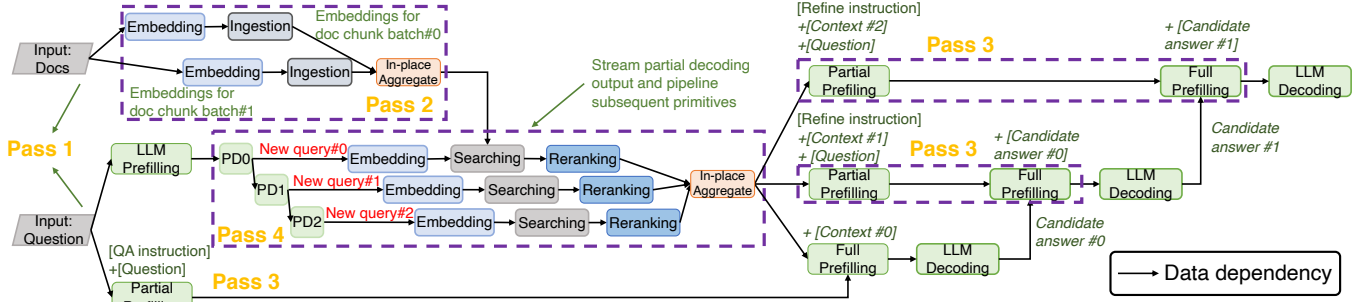


图 6. 高级 RAG 文档问答、采用 *refine* 答案合成模式的优化 e-graph 示例。PD 为部分解码；Partial / Full Prefilling 注明正在计算的提示词部分，原语元数据省略，块长不代表执行时间。Input Docs / Question 为输入文档 / 问题；Embedding / Ingestion / Searching / Reranking 为嵌入 / 写入 / 搜索 / 重排，LLM Prefilling / Decoding 为预填充 / 解码，In-place Aggregate 为就地聚合。QA instruction 为问答指令，Refine instruction 为改进答案的指令，Context 为上下文片段，Candidate answer 为候选答案，New query 为扩展查询；doc chunk batch 为文档片段批次。Stream partial decoding output and pipeline subsequent primitives 表示将部分解码输出流式送往下游形成流水线；Data dependency 为数据依赖，Pass 1-4 为四项优化步骤。

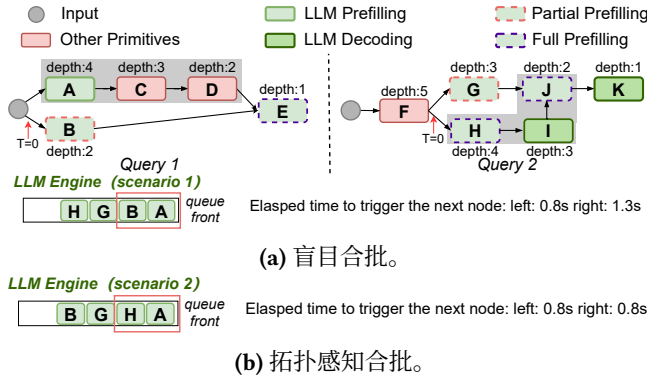


图 7. 最大词元容量为 1024 的 LLM 实例上，两种合批方式的示例。假设每个普通或部分 / 剩余预填充输入为 512 词元；单个耗时 0.5 秒，两个 512 词元输入合批耗时 0.8 秒。Input 为输入，Other Primitives 为其他原语，LLM Prefilling / Decoding 为预填充 / 解码，Partial / Full Prefilling 为部分 / 剩余完整预填充，Query 1 / 2 为查询 1 / 2，depth 为距输出的拓扑深度，queue front 为队首；scenario 1 / 2 为方案 1 / 2。下方文字表示触发下一节点的时间：盲目合批为左 0.8 秒、右 1.3 秒；拓扑感知合批为两侧均 0.8 秒。

时，例如图 2a 的代理与判定模型，或图 2d 的查询扩展与答案合成共用 LLM。

基础方案及局限：盲目合批。朴素方法把不同原语统一处理。引擎调度器以 FIFO 从等待队列动态合批，达到预设最大批量或等待超时，派发实例执行，类似已有系统 [22, 29]。但同一查询的不同节点，对图进展的贡献并不相等。

如图 7，查询 1 的 A、B 与查询 2 的 G、H 一起进入 LLM 队列。盲目合批会选择 A、B，使 G、H 等待。然而，B 的子节点 E 还受未触发的另一父节点 D 限制，此时执行 B 收益较小。若改为合批 A、H，两个查询的图都可前进，推迟 B 不会成为查询 1 的瓶颈。

我们的方案：拓扑感知合批。示例揭示，盲目合批忽略不同原语对图进展的贡献。原语拓扑深度不同，适当推

算法 2 拓扑感知合批

- 1: 事件一：取得查询优化后的执行图 $\mathcal{G}$ ：
按逆拓扑顺序确定节点深度。
- 2: $\mathcal{G}' \leftarrow \text{RevTopoSort}(\mathcal{G})$; $\text{InitDepth}(\mathcal{G}')$
- 3: 对 $v \in \mathcal{G}'$ 执行
- 4: 对 $p \in v.\text{parents}$ 执行
- 5: $p.\text{depth} \leftarrow \max(p.\text{depth}, v.\text{depth} + 1)$
- 6: 事件二：引擎调度器的调度周期：
- 7: # 根据原语深度与关系组成批次。
- 8: $\text{max_bs} \leftarrow \text{GetConfigBatchsize}()$; $\text{batch} \leftarrow []$
- 9: $\mathcal{B} \leftarrow$ 将队列中同一查询的节点分桶，按各桶最早节点到达时刻排序。
- 10: 对 $b \in \mathcal{B}$ 执行
- 11: $\text{slots} \leftarrow \text{max_bs} - \text{batch.size}()$
- 12: 若 $\text{slots} = 0$ 则
- 13: 跳出循环
- 14: $\text{candidates} \leftarrow$ 从 b 中深度最大的各节点取出关联请求，最多 slots 个。
- 15: 移除关联请求全部已调度的节点。
- 16: $\text{batch.append}(\text{candidates})$

迟深度较小节点，可为贡献更大的节点预留资源。还需考虑请求之间的关联与依赖 (§2.3)。据此，我们提出一种启发式方案，利用原语深度及其关系，指导批次形成。

它具有两方面收益。第一，同一查询的深度信息自然包含原语依赖，结合每个原语内部请求关联，可调整调度偏好 (§2.3)。例如，相同深度原语可使用最大高效批量，优化吞吐量、推动执行图。第二，实际时延不可预测，深度不能准确定位关键路径，但可指导查询内部优先级 (图 7)，更有效地在多个查询间利用资源。

算法 2 给出流程。先逆拓扑排序并记录节点深度，输出节点深度最小，对应事件一。调度时，将同一查询的排队原语放入同桶，桶内按深度排序，较深节点优先；各桶再按其中最早节点到达时刻排序。槽位数对应预先设定的最大批量，LLM 则对应最大词元容量，以保持最

佳吞吐效率。调度器逐桶检查最高优先级节点，有空槽便将其关联请求放入候选批次，对应事件二。⁵

5.3 应用共置

Teola 的设计目标是优化单个应用 workflow。多个应用共享基础设施时 [35, 78]，仍由系统独立编排：它们的执行图可能包含相同或不同原语，携带不同应用级与查询级元数据，但处理方式与单应用相同。这没有充分利用进一步优化机会，例如应用共享同一 LLM 引擎和模型时，跨应用 KV 缓存共享还可提高吞吐量。我们计划未来研究。为完整起见，在 §7.2 评估并讨论共置应用的处理能力。

6 实现

Teola 原型包含约 ~5300 行 Python。使用的库包括：(1) Ray [61]，实现分布式调度与执行；(2) LlamaIndex [1]，处理文本分片、HTML / PDF 解析等预处理；(3) PostgreSQL [20]，作为默认数据库；(4) pgvector [19]，执行向量检索；(5) Google Custom Search [9]，提供单个及批量搜索；(6) vLLM [48]，作为模型服务引擎，并修改以支持表 2 的部分预填充与剩余完整预填充。

前端通过 FastAPI [7] 提供提交查询与配置的用户接口。后端图调度器维护线程池，为每个新查询分配专属线程，构建、优化并派发执行图。除 §5 所述机制，各引擎调度器还在实例间做负载均衡；普通引擎主要依据已执行请求数量，LLM 主要依据已占用 KV 缓存槽位。

减少通信开销。对数据交互较大或使用同一引擎的相邻原语，集中式调度器采用依赖预调度。它同时发出相互依赖的 A、B，让 B 在后端等待 A 的结果。A 完成后，除向调度器报告结果外，RPC 还将输出直接发给 B 的引擎，避免先经调度器中转，再派发 B，从而减少通信开销。

7 评估

平台配置。模型引擎分配 GPU，无模型引擎使用纯 CPU 资源。嵌入和其他非 LLM 模型各实例运行于一张 NVIDIA 3090 24GB。gemma-2-2B、llama-2-7B、llama-2-13B [69] 各实例分别使用 1、1、2 张 3090；llama-30B [68] 各实例使用两张 NVIDIA A800 80GB。物理服务器间网络带宽为 100 Gbps。

基线。据我们所知，专门优化分布式 LLM 应用 workflow 执行的研究较少，因此比较以下流行框架及变体：

- **LlamaDist**：基于 Ray 的 LlamaIndex 分布式实现，原框架当时有超过 36000 个 GitHub star。它以任务模块链构建应用，各模块调用不同分布式后端请求。使用与 Teola 相同引擎，区别主要在编排粒度。
- **LlamaDistPC (并行与缓存复用)**：增强版 LlamaDist，分析预定义管线，手工并行化独立模块，并用已有前缀缓存技术 [33, 52, 54, 79]，避免部分提示词指令重算。

⁵译注：原代码标识符保留。RevTopoSort / InitDepth 为逆拓扑排序 / 初始化深度，parents 为父节点，GetConfigBatchsize 为读取容量配置，candidates 为候选请求，slots 为剩余槽位。伪代码以 batch.size 计算槽位数，而正文同时讨论 LLM 的词元容量，未明确说明按请求数与按词元数计费的转换；实现具体合批时仍需满足相应资源单位。深度是到输出的拓扑距离，不是累计执行时长，也不是概率关键路径估计。

- **AutoGen**：当时有超过 32000 个 star 的流行框架。它应用构建多个智能体，各管理若干系统模块，例如检索智能体管理索引、查询嵌入和搜索；智能体按预定义图通信。

已部署引擎的请求调度比较两种方式：

- **单次调用优先 (PO)**：稍作修改，将一次编排调用的请求打包，实质为基线补充 Teola 利用的关联信息。引擎一次调度一个请求包，优先优化每次调用时延。
- **吞吐量优先 (TO)**：预先为引擎调优最大批量或词元容量，按 2 的幂次增加 DNN 批量或 LLM 词元数，直到吞吐量不再提高；采用动态合批 [22, 29, 48]，最大化总吞吐量，但忽略请求关系。

应用、模型与负载。原文称实验覆盖三种应用，实际列出如下四类：⁶

- **搜索辅助生成 (图 2a)**：搜索引擎辅助核心 LLM 回答。较小的 llama-2-7B 作为代理与判定模型，给出初步回答并决定是否搜索；前四个搜索结果送入核心模型合成答案。使用 web_question [28] 和 HotpotQA [75] 构造请求，按泊松过程合成到达时刻。
- **朴素 RAG 文档问答 (图 2c)**：用户输入文档或网页及问题，系统按默认大小 256、重叠 30 分片，使用 bge-large-en-v1.5 [74] 生成嵌入，写入 PostgreSQL 与 pgvector 向量数据库。默认检索前三个相关片段，通过树式合成生成回答。问题及文档 / 网页取自 Finqabench [8] 与 TruthfulQA [53]，按泊松过程构造负载。
- **高级 RAG 文档问答 (图 2d)**：在朴素版本上增加核心 LLM 查询扩展，默认生成三个改写查询。bge-reranker-large [74] 评估相关性，每个查询检索 16 个片段，再选出全局前三个，使用 §4.2 的 refine 模式生成答案。
- **上下文检索 (图 2e)**：在朴素 RAG 上，为索引前每个片段加入轻量模型 gemma-2-2B [67] 上下文。为减少时间与成本，每片结合四个邻近片段，生成简短上下文并加到片段前。与高级 RAG 相同的模型重排检索到的 32 个片段，前三个送入核心 LLM，一次生成最终答案。

除非另有说明，使用上述默认配置，模型均为半精度，并测试 7B、13B 和 30B 核心 LLM。

7.1 端到端性能

我们在相同资源配置下，比较不同方案运行各类应用的性能。每个非 LLM 执行引擎部署一个实例，每个 LLM 则部署两个实例。

搜索引擎增强生成。图 8 第一行表明，Teola 相对其他方案最多获得 1.79 倍加速。其效率来自两方面：评判模型和核心 LLM 可以并行完成指令与问题部分的预填充；不同引擎之间的批处理也能得到有效协调。LlamaDist 按顺序执行模块，而且难以有效调度多个查询产生的请求。PO 侧重降低单次调用延迟，因此在高请求率下会产生较长的排队时间；TO 在这类场景中通常表现更好。LlamaDistPC 没有显式实现跨模块并行，因此无法充分受益于并行化。它只缓存部分指令的前缀，这些前缀通

⁶译注：保留这一正文与列表数量差异；总性能图也有四行，对应四类 workflow。

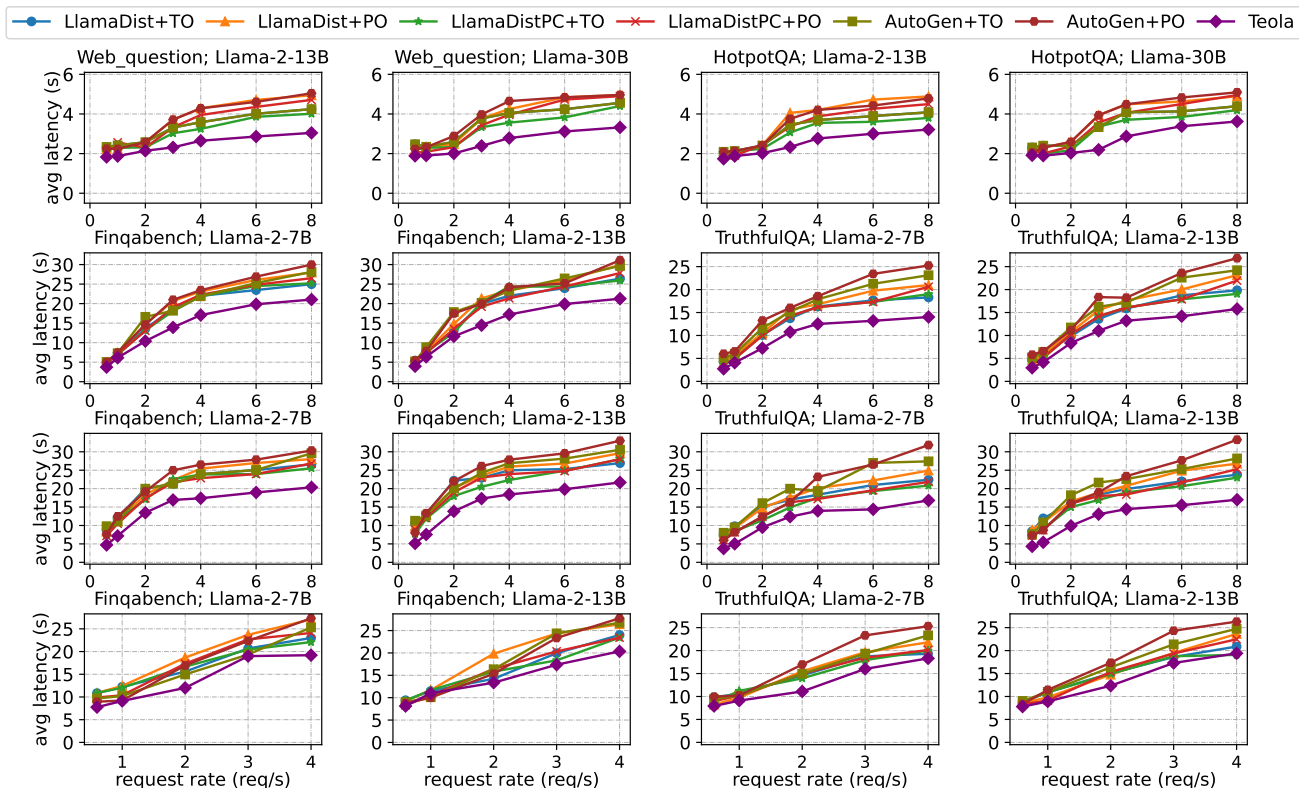


图 8. 搜索辅助生成（第一行）、朴素 RAG 文档问答（第二行）、高级 RAG 文档问答（第三行）与上下文检索（第四行）的端到端性能。子图标题给出数据集与核心 LLM；横轴 request rate 为请求率（请求/秒），纵轴 avg latency 为平均时延（秒）。曲线比较 LlamaDist、LlamaDistPC 和 AutoGen，各搭配 TO（吞吐量优先）或 PO（单次调用优先）调度，以及 Teola。

常约有 60 个 token，收益有限；前缀显著更长时，前缀缓存才更有利 [2, 79]。在该应用中，AutoGen 分别为代理模型、评判模型、搜索引擎和 LLM 合成器定义智能体，形成与 LlamaDist 类似的结构，性能因而也相近。⁷

采用基础 RAG 的文档问答。图 8 第二行显示，Teola 在低请求率下最多获得 1.62 倍加速，在高请求率下最多获得 1.67 倍加速。LlamaDist 顺序执行模块，没有专门优化；LlamaDistPC 则使索引构建与查询嵌入模块有限并行，并复用部分指令的 KV 缓存，因此略优于 LlamaDist。AutoGen 的工作流由检索智能体和合成智能体组成，在请求量较小时表现尚可，但其较简单的编排架构在高负载下会造成较长的排队延迟。从调度策略看，PO 注重单次调用延迟，故在低请求率下优于 TO，但高请求率下性能较差。此外，该应用还存在较复杂的请求关系：索引构建与查询嵌入模块均使用嵌入模型；LLM 合成模块先发起三个请求，再发起后续请求，完成树形合成。忽略这些关系，会像 TO 一样造成批处理效率降低和有效吞吐量下降。Teola 则利用执行图，把文档分块的大规模嵌入等计算密集任务拆成流水线阶段，并挖掘更多并行机

会，例如四次部分预填充。其感知拓扑的批处理还捕捉不同原语请求之间的依赖与关联，帮助各引擎有效组批。

采用高级 RAG 的文档问答。该应用是实验中最复杂的应用，也提供了充分展示 Teola 优化效果的机会。图 6 所示的优化包括多个层次的并行化，例如并行运行独立的数据流分支和不同 LLM 调用的部分预填充；还包括流水线化，例如把大批量嵌入拆成小批量，以及把查询扩展的解码过程拆成三个部分解码阶段。LlamaDist 采用简单的运行至完成方式，顺序执行 workflow，错过了降低端到端延迟的机会。LlamaDistPC 改善了索引构建与查询扩展模块之间的并行，并复用部分 KV 缓存，但仍未像 Teola 那样充分挖掘优化空间。AutoGen 为检索、重排序、查询扩展及 LLM 合成定义四个智能体，但难以有效实现流水线与并行操作，因此在高请求负载下表现不佳。此外，与基础 RAG 的情况类似，无论采用 PO 还是 TO，其他基线都难以有效协调请求，而 Teola 表现良好。总体而言，Teola 在低请求率下最多获得 2.09 倍加速，在高请求率下最多获得 2.03 倍加速，如图 8 第三行所示。

上下文检索。与基础 RAG 相比，这个工作流主要增加了两个环节：索引构建前，使用轻量 LLM 为分块补充上下文；搜索后，进行重排序。由于待处理分块数量很大，所有方案的大部分时间都消耗在上下文补充上。这部分可用的图优化机会较少，但 Teola 对应用的感知调

⁷译注：原文这里以及下面的基础 RAG 段落称“其他四种方案”，但实验比较包括多个框架各自的 PO 与 TO 配置；图中具体比较对象应以图例为准。加速倍数是这些实验中的基线延迟与 Teola 延迟之比。

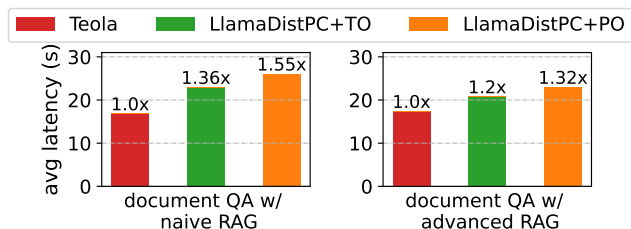


图 9. 应用共置场景下两个应用的平均延迟。核心 LLM 为 llama-2-13B，数据集为 TruthfulQA。图中两个应用分别为基础 RAG 和高级 RAG；纵轴为平均延迟，单位为秒。各柱对应 Teola、采用 TO 的 LlamaDistPC 和采用 PO 的 LlamaDistPC，标注的倍数表示相对 Teola 的延迟比。

度，以及对其他组件的图优化，仍使其相对其他基线获得 1.06–1.59 倍加速，如图 8 第四行所示，进一步体现了系统的适用性。

7.2 应用共置

除单应用场景外，我们还评估 Teola 并发处理多个应用查询的能力。具体而言，一个采用基础 RAG 的文档问答应用，与一个采用高级 RAG 的文档问答应用，在同一底层基础设施上同时运行。它们使用相似的后端引擎，但调用语义不同：高级 RAG 文档问答还需要一个重排序引擎。

我们以较强的基线 LlamaDistPC 为比较对象，每个应用的请求率均为每秒 3 个请求。图 9 给出两个应用各自查询的平均延迟。结果显示，在多应用共置场景下，Teola 仍保持对 LlamaDistPC 的性能优势，对这两个应用获得 1.2–1.55 倍延迟加速。这说明其处理多个应用查询的能力较为稳定，原因在于其图优化方法不依赖具体应用，而调度方法能够感知应用。

实际部署中，各应用还可能具有不同优先级。例如，LLM 聊天服务倾向于缩短延迟以改善用户体验，而文档摘要服务可以容忍更长的延迟。这些优先级差异会影响基础设施处理请求的顺序。不同应用的目标可以编码为所生成原语的元数据，供各引擎调度器进行基于优先级或资源约束的调度。不过，本文不进一步研究这一问题。

7.3 消融实验

我们从图优化和运行时调度两个方面，验证 Teola 主要组件的有效性。

图优化。图 10 比较了不同优化配置下 Teola 的性能，即分别启用或关闭并行化优化（第 1、3 遍）与流水线化优化（第 2、4 遍），具体优化见 §4。左图给出了单查询平均延迟，明确表明并行化和流水线化都能有效利用优化机会、降低延迟。在包含不同请求率的请求轨迹上，这一结果同样成立。

运行时调度。图 11 展示了启用或关闭 §5.2 所述感知拓扑的批处理对性能的影响。左图说明，这种方法能识别同一引擎上不同原语的深度差异，并依据已知的关联和依赖关系安排调度，从而改善应用层性能。它使单查询执行平均获得 1.15 倍加速。在多查询场景下，感知拓扑

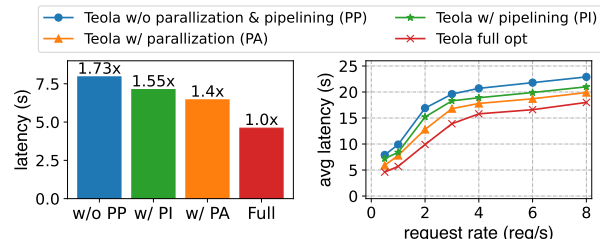


图 10. 图优化消融实验：在 TruthfulQA 数据集上运行高级 RAG 文档问答，核心 LLM 为 llama-30B。左图为单查询运行 10 次后的平均延迟；右图为不同请求负载下的平均延迟。图中 PP 表示并行化与流水线化，PA 表示并行化，PI 表示流水线化；“w/o PP”为二者均关闭，“w/ PI”为仅开启流水线化，“w/ PA”为仅开启并行化，“Full”为完整优化。纵轴为延迟或平均延迟，单位为秒；右图横轴为每秒请求数。

部分预填充	后续完整预填充	合计	单次完整预填充
76.03 (200)	215.89 (800)	291.92 (1000)	260.36 (1000)
217.67 (850)	222.66 (850)	440.33 (1700)	414.09 (1700)
582.95 (2500)	159.65 (500)	742.60 (3000)	720.15 (3000)

表 3. llama-2-7B 在不同输入规模下的执行效率比较：左侧为 Teola 拆分后的预填充，即部分预填充与后续完整预填充；右侧为一次性完成的完整预填充。数值表示执行时间，括号内为输入 token 数。译注：原文时间单位写作“millionsecond”，属于拼写错误；结合表中预填充耗时，这里的单位应为毫秒。后续阶段复用先前部分预填充的状态，括号中的阶段输入规模相加为总输入规模。

的批处理仍然有效：除提高单查询效率外，它还把不同查询中有助于推进执行的原语有效融合到批次中，促进整体执行，最多降低 19.2% 的平均延迟。

7.4 开销分析

以上结果表明，Teola 能显著改善现有方案的性能。为进一步理解它引入的开销，我们从图优化、数据通信和执行效率三个方面进行分析。首先，每个查询的执行图优化会带来额外开销，但 LLM 工作流的原语数量有限，而且同一应用的查询可复用缓存的优化结果，因此这部分开销通常很小。其次，相比现有粗粒度方案，原语之间的通信会产生更多开销。Teola 的预调度机制可以缓解这一问题；流水线执行还允许下游更早处理已经传来的部分数据。最后，把某些操作拆分，例如拆分 LLM 预填充，可能比一次性完成整个操作更高效，原因包括反复将 KV 缓存搬入 SRAM，或无法充分利用 GPU。不过，这种效率损失较小：表 3 的比较表明，耗时仅增加 3.11%–12.12%。新增的并行机会进一步超过了这部分小幅损失，带来更大的端到端加速。

我们以 TruthfulQA 数据集上的高级 RAG 文档问答为例，分析延迟构成。使用缓存优化时，图优化开销只占总延迟的 1.3%–3%，通信开销也较低，仅占 3.1%–6.2%。在资源固定而请求率提高的情况下，排队延迟逐渐主导端到端时间，因此这些开销在整体执行时间中的相对占比还会进一步降低。

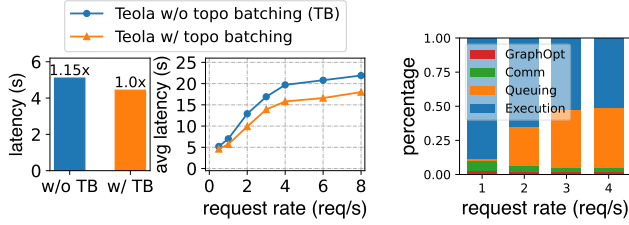
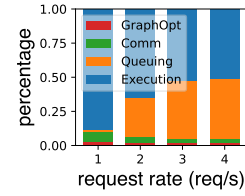


图 11. 运行时调度消融实验。实验设置与图 10 相同。TB 表示感知拓扑的批处理；“w/o TB”为关闭，“w/ TB”为开启。左图比较单查询延迟，右图比较不同请求率下的平均延迟；时间单位为秒，请求率单位为每秒请求数。

图 12. Teola 执行关键路径的延迟构成。横轴为每秒请求数，纵轴为各类时间所占比例。图例依次为图优化、通信、排队和实际执行。



8 局限与未来工作

本节指出 Teola 的局限与未来研究方向。

动态工作流。 LLM 工作流通常由人预先设计，性能稳定，因此广泛用于开源项目 [1, 11, 12, 14] 和生产环境 [10, 15, 17]。Teola 主要针对这类结构化场景。然而，也出现了无需人工干预、由 LLM 动态规划并执行操作的自主智能体，没有固定工作流。Teola 的提前图优化难以处理这类场景，尤其是自我反思 [58]，因为执行前无法得到完整原语图。适应动态工作流是很有前景的研究方向。

与后端的耦合。 为实现细粒度编排，我们必须修改部分引擎机制，例如支持拆分原语和特定合批策略。相比编排与执行解耦、直接使用可插拔引擎的框架 [1, 12]，这需要额外工程工作，是性能与模块化之间的必要权衡。对用户而言，这些优化细节仍封装在系统内部。

利用关键路径。 执行图的关键路径信息还可进一步使用。资源分配时，可依据负载模式调整关键与非关键路径操作的资源，提高利用率；调度时，可优先服务特定查询的关键节点，增强现有拓扑合批，但需要准确在线预测关键路径，并处理协调复杂性。

9 相关工作

LLM 推理优化。 已有大量研究关注内核加速 [30, 36, 73]、请求调度 [24, 25, 65, 76]、模型并行 [50, 60, 80]、语义缓存 [26, 81]、KV 缓存管理 [48, 51, 71]、KV 缓存复用 [33, 45, 48, 55, 79] 和先进解码算法 [56, 59, 62]。近期研究 [39, 63, 80] 分离部署预填充与解码，提高有效吞吐量，这与 Teola 的拆分思想一致，可结合使用。多数工作提供 LLM 领域的单点方案，而 Teola 从整体视角优化完整应用工作流，促进不同组件协作，因此推理优化与本系统互补。

LLM 应用框架。 除前述框架 [1, 2, 12]，部分研究 [46, 47, 52, 79] 使用专门接口或编译器，优化包含多次 LLM 调用的复杂任务，探索并行及 KV 缓存共享。智能体框架 [13, 37, 38, 64] 让模型决策、选工具和相互交互，减少人工干预，也带来特定挑战。Teola 更接近具有多组件、预定义工作流的框架 [1, 2, 12]，专注端到端效率。

Parrot [52] 也捕获多次调用的应用级亲和关系并联合调度；SGLang [79] 探索请求关系和历史 KV 复用。它们同样关注单次推理之外的机会。区别在于，Teola 优化包含 LLM 与非 LLM 组件的完整应用级工作流，这一部分此前未被同样考虑。

数据分析系统。 这些系统 [27, 34, 42, 77] 通过基于图的依赖分析，优化大数据分布式任务执行。Teola 借鉴这些图优化方法；但它们面向通用数据处理，Teola 则为 LLM 工作流提出专门抽象，支持图优化及本文所述方法之外的更大优化空间。

10 结论

我们提出细粒度 LLM 应用编排框架 Teola，核心是原语级数据流图。它显式暴露原语属性及交互，自然支持工作流级并行执行优化。利用图中的原语关系，Teola 通过拓扑感知合批启发式，将原语请求合理融合执行。平台实验表明，在不同应用上，本系统优于现有比较方案。

参考文献

- [1] LlamaIndex. https://github.com/jerryliu/llama_index, 2022.
- [2] Promptflow. <https://github.com/microsoft/promptflow>, 2023.
- [3] Autogpt. <https://github.com/Significant-Gravitas/AutoGPT>, 2024.
- [4] Bing Copilot. <https://www.bing.com/chat>, 2024.
- [5] Character.ai/. <https://character.ai/>, 2024.
- [6] contextual-retrieval. <https://www.anthropic.com/news/contextual-retrieval>, 2024.
- [7] Fastapi. <https://fastapi.tiangolo.com/>, 2024.
- [8] Finqabench Dataset. <https://huggingface.co/datasets/lighthouai/finqabench>, 2024.
- [9] Google custom search. <https://programmablesearchengine.google.com/>, 2024.
- [10] Gpt-rag. <https://github.com/Azure/GPT-RAG>, 2024.
- [11] haystack. <https://github.com/deepset-ai/haystack>, 2024.
- [12] Langchain. <https://github.com/langchain-ai/langchain>, 2024.
- [13] LangGraph. <https://python.langchain.com/docs/langgraph/>, 2024.
- [14] Lazyllm. <https://github.com/LazyAGI/LazyLLM>, 2024.
- [15] Observability of llm applications: Exploration and practice from the perspective of trace. https://www.alibabacloud.com/blog/observability-of-llm-applications-exploration-and-practice-from-the-perspective-of-trace_601604, 2024.
- [16] Openai function calling. <https://platform.openai.com/docs/guides/function-calling>, 2024.
- [17] Pairag. <https://github.com/aigc-apps/PAI-RAG>, 2024.
- [18] Perplexity ai. <https://www.perplexity.ai/>, 2024.
- [19] Pgvector. <https://github.com/pgvector/pgvector>, 2024.
- [20] Postgresql. <https://www.postgresql.org/>, 2024.
- [21] Privatellm. <https://privatellm.app/en>, 2024.
- [22] Triton inference server. <https://github.com/triton-inference-server>, 2024.
- [23] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A system for Large-Scale machine learning. In *Proc. USENIX OSDI*, 2016.
- [24] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve. *arXiv preprint arXiv:2403.02310*, 2024.
- [25] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.
- [26] Fu Bang. Gptcache: An open-source semantic cache for llm applications enabling faster answers and cost savings. In *Proc. the 3rd Workshop for Natural Language Processing Open Source Software*, 2023.
- [27] Michael Bauer, Sean Treichler, Elliott Slaughter, and Alex Aiken. Legion: Expressing locality and independence with logical regions. In *Proc. IEEE SC*, 2012.
- [28] Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. Semantic parsing on Freebase from question-answer pairs. In *Proc. EMNLP*, October 2013.
- [29] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. Clipper: A {Low-Latency} online prediction serving system. In *Proc. USENIX NSDI*, 2017.
- [30] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Proc. NeurIPS*, 2022.
- [31] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proc. ACL*, 2018.
- [32] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [33] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khadelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.
- [34] Ionel Gog, Malte Schwarzkopf, Natacha Crooks, Matthew P Grosvenor, Allen Clement, and Steven Hand. Musketeer: all for one, one for all in data processing systems. In *Proc. ACM Eurosys*, 2015.
- [35] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving {DNNs} like clockwork: Performance predictability from the bottom up. In *Proc. USENIX OSDI*, 2020.
- [36] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Hanyu Dong, and Yu Wang. Flashdecoding++: Faster large language model inference on gpus. In *Proc. Machine Learning and Systems*, 2023.
- [37] Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, Li Zhang, Lingyao Zhang, Min Yang, Mingchen Zhuge, Taicheng Guo, Tuo Zhou, Wei Tao, Wenyi Wang, Xiangru Tang, Xiangtao Lu, Xiawu Zheng, Xinbing Liang, Yaying Fei, Yuheng Cheng, Zongze Xu, and Chenglin Wu. Data interpreter: An llm agent for data science. *arXiv preprint arXiv:2402.18679*, 2024.
- [38] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [39] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, et al. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.
- [40] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.
- [41] Zhongzhen Huang, Kui Xue, Yongqi Fan, Linjie Mu, Ruoyu Liu, Tong Ruan, Shaoting Zhang, and Xiaofan Zhang. Tool calling: Enhancing medication consultation via retrieval-augmented large language models. *arXiv preprint arXiv:2404.17897*, 2024.
- [42] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: distributed data-parallel programs from sequential building blocks. In *Proc. ACM Eurosys*, 2007.
- [43] Rolf Jagerman, Honglei Zhuang, Zhen Qin, Xuanhui Wang, and Michael Bendersky. Query expansion by prompting large language models. *arXiv preprint arXiv:2305.03653*, 2023.
- [44] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv preprint arXiv:2403.14403*, 2024.
- [45] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*, 2024.
- [46] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- [47] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. *arXiv preprint arXiv:2312.04511*, 2023.

- [48] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proc. ACM SOSP*, 2023.
- [49] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2020.
- [50] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In *Proc. USENIX OSDI*, 2023.
- [51] Bin Lin, Tao Peng, Chen Zhang, Minmin Sun, Lanbo Li, Hanyu Zhao, Wencong Xiao, Qi Xu, Xiafei Qiu, Shen Li, et al. Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache. *arXiv preprint arXiv:2401.02669*, 2024.
- [52] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of llm-based applications with semantic variable. In *Proc. USENIX OSDI*, 2024.
- [53] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. *arXiv preprint arXiv:2109.07958*, 2021.
- [54] Shu Liu, Asim Biswal, Audrey Cheng, Xiangxi Mo, Shiyi Cao, Joseph E Gonzalez, Ion Stoica, and Matei Zaharia. Optimizing llm queries in relational workloads. *arXiv preprint arXiv:2403.05821*, 2024.
- [55] Shu Liu, Asim Biswal, Audrey Cheng, Xiangxi Mo, Shiyi Cao, Joseph E Gonzalez, Ion Stoica, and Matei Zaharia. Optimizing llm queries in relational workloads. *arXiv preprint arXiv:2403.05821*, 2024.
- [56] Xiaoxuan Liu, Lanxian Hu, Peter Bailis, Ion Stoica, Zhijie Deng, Alvin Cheung, and Hao Zhang. Online speculative decoding. *arXiv preprint arXiv:2310.07177*, 2023.
- [57] Yanming Liu, Xinyue Peng, Xuhong Zhang, Weihao Liu, Jianwei Yin, Jiannan Cao, and Tianyu Du. Ra-isf: Learning to answer and understand from retrieval augmentation via iterative self-feedback. *arXiv preprint arXiv:2403.06840*, 2024.
- [58] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Proc. NeurIPS*, 2024.
- [59] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proc. ACM ASPLOS*, 2024.
- [60] Xupeng Miao, Chunlan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. Spotserve: Serving generative large language models on preemptible instances. In *Proc. ACM ASPLOS*, 2024.
- [61] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. Ray: A distributed framework for emerging {AI} applications. In *Proc. USENIX OSDI*, 2018.
- [62] Jie Ou, Yueming Chen, and Wenhong Tian. Lossless acceleration of large language model via adaptive n-gram parallel decoding. *arXiv preprint arXiv:2404.08698*, 2024.
- [63] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. *arXiv preprint arXiv:2311.18677*, 2023.
- [64] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in huggingface. In *Proc. NeurIPS*, 2023.
- [65] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. Fairness in serving large language models. *arXiv preprint arXiv:2401.00588*, 2023.
- [66] Jiejun Tan, Zhicheng Dou, Yutao Zhu, Peidong Guo, Kun Fang, and Ji-Rong Wen. Small models, big insights: Leveraging slim proxy models to decide when and what to retrieve for llms. *arXiv preprint arXiv:2402.12052*, 2024.
- [67] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- [68] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [69] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shrutit Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [70] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In *Proc. NeurIPS*, 2017.
- [71] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. *arXiv preprint arXiv:2404.09526*, 2024.
- [72] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Auto-gen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.
- [73] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *Proc. ICML*, 2023.
- [74] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighof. C-pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.
- [75] Zhilin Lin, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proc. EMNLP*, 2018.
- [76] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *Proc. USENIX OSDI*, 2022.
- [77] Matei Zaharia, Reynold S Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shriram Venkataraman, Michael J Franklin, et al. Apache spark: a unified engine for big data processing. *Communications of the ACM*, 2016.
- [78] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. {SHEPHERD}: Serving {DNNs} in the wild. In *Proc. USENIX NSDI*, 2023.
- [79] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Efficiently programming large language models using sglang. *arXiv preprint arXiv:2312.07104*, 2023.
- [80] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. *arXiv preprint arXiv:2401.09670*, 2024.
- [81] Banghua Zhu, Ying Sheng, Lianmin Zheng, Clark Barrett, Michael I Jordan, and Jiantao Jiao. On optimal caching and model multiplexing for large model inference. *arXiv preprint arXiv:2306.02003*, 2023.